

ECOM EXPERTS

AyudaPS — Documentación PrestaShop

Manual de usuario e instalación

ayudaps 1.0.0 · 04/09/2026

Contenido

1. Qué hace y para quién

2. Compatibilidad

3. Instalación

4. Configuración

Lo que hay que tocar sí o sí

Lo que conviene revisar

La clave de la herramienta de caché

El fichero .user.ini

5. Cómo se usa, día a día

Publicar una guía nueva

Publicar una novedad

Añadir un hook

Las tres herramientas interactivas

El buscador

6. Qué ve el visitante

7. Preguntas frecuentes y problemas

8. Seguridad

Lo que se ha arreglado en esta revisión

Fallos de funcionamiento encontrados y arreglados

Lo que se ha revisado y estaba bien

Lo que queda pendiente

Cómo se ha comprobado

9. Ficha técnica

Estructura

Rutas públicas

Rutas bloqueadas

Base de datos, cron y permisos

Guiones de línea de comandos

Ficheros tocados en esta revisión

Manual de entrega. Todo lo que hay aquí está comprobado sobre los ficheros del proyecto y sobre el sitio arrancado en local con el servidor de PHP.

1. Qué hace y para quién

AyudaPS no es un módulo de PrestaShop: es un sitio web de documentación. Se instala en un servidor propio, no dentro de una tienda, y no toca la base de datos de nadie. Su trabajo es reunir en un único sitio, bien organizado y buscable, todo lo que un desarrollador necesita saber para trabajar con PrestaShop 1.6, 1.7, 8.x y 9.x: los hooks, cómo se hace un módulo, cómo funciona un tema, qué se puede y qué no se puede hacer con un override, cómo va el webservice, y un largo etcétera.

El problema que resuelve es el de siempre en este ecosistema: la documentación oficial de PrestaShop está repartida entre varios sitios, mezcla versiones sin avisar, y buena parte de lo que hace falta de verdad —el nombre exacto de un hook, en qué versión apareció, qué parámetros recibe— no está escrito en ningún lado y hay que sacarlo leyendo el núcleo. Aquí está recogido, en castellano, con el ejemplo de código al lado y una etiqueta que dice desde qué versión existe cada cosa.

Va dirigido a tres perfiles. Al **desarrollador de módulos**, que entra buscando un hook concreto o un patrón (HelperForm, Grid, ObjectModel, Doctrine) y se lleva el código listo para pegar. Al **administrador de la tienda**, que tiene dos secciones enteras escritas para él —«Guía Back Office» y «Guía Front Office»— explicando cómo se llevan los pedidos, el catálogo, los transportistas o el checkout sin necesidad de tocar código. Y a los **asistentes de IA**: cada página tiene una versión en Markdown descargable pensada para que un modelo de lenguaje la lea sin el ruido del HTML.

Técnicamente es un sitio en PHP plano, sin base de datos, sin Composer y sin Node. Todo el contenido son ficheros JSON en `data/`, y el sitio los convierte en páginas HTML al vuelo con varias capas de caché por delante. Eso lo hace muy rápido, muy barato de alojar y muy fácil de mantener: para publicar una guía nueva basta con dejar un JSON en su carpeta.

2. Compatibilidad

PHP	8.0 mínimo. Probado y sin un solo aviso en 8.2. Sintaxis validada también en 8.4.
PHP 7.4	No funciona. Ver el aviso de más abajo.
Servidor web	Apache 2.4 con <code>mod_rewrite</code> y <code>AllowOverride All</code> . Nginx sirve igual: la configuración de ejemplo viene comentada al final del <code>.htaccess</code> .
Base de datos	Ninguna. El contenido son ficheros JSON.
Dependencias	Ninguna. Ni Composer, ni <code>vendor/</code> , ni Node, ni npm. Solo PHP.
Extensiones PHP	Solo las de serie (<code>json</code> , <code>pcre</code>). <code>redis</code> , <code>apcu</code> y <code>opcache</code> son opcionales : si están, se usan; si no, la caché cae a fichero sola.
Memoria PHP	64 MB sobran. El <code>.user.ini</code> la fija ahí.
Disco	Unos 50 MB con el <code>export/</code> generado.
Idioma	Castellano, un único idioma. No hay sistema de traducción ni está previsto.
PrestaShop	No hace falta ninguna instalación de PrestaShop. El sitio <i>habla</i> de PrestaShop, no se integra con él.

⚠ PHP 7.4 no vale, aunque el `INSTALL.md` original dijera 7.4+. Comprobado con `php -l`:
`src/Renderer/ContentRenderer.php` usa `match()` y `src/Search/SearchEngine.php` usa `str_contains()`
y `str_starts_with()`, tres cosas que solo existen desde PHP 8.0. En 7.4 el sitio se cae con un error de sintaxis en cuanto pinta una página con contenido. El requisito ya está corregido en el `INSTALL.md` del propio proyecto.

Multitienda / multi-instalación. No aplica: es un sitio, no un módulo. Si se quiere publicar en un subdirectorio (<https://midominio.com/docs/>) está previsto: se cambia `SITE_SUBDIR` en `config.php` y `RewriteBase` en el `.htaccess`, y todos los enlaces y llamadas AJAX se recalculan solos.

3. Instalación

No hay instalador ni asistente. Son cuatro pasos:

1. **Subir los ficheros** al directorio raíz del dominio (o del subdirectorio).
2. **Comprobar el `.htaccess`**. Apache tiene que tener `mod_rewrite` activo y `AllowOverride All` en la carpeta. Sin eso, las URLs limpias (`/hooks/actionCartSave`) no funcionan y todo devuelve 404.
3. **Ajustar `config.php`** — como mínimo `SITE_DOMAIN`, y `SITE_SUBDIR` si no está en la raíz. Ver la sección 4.

4. **Permisos de escritura** en dos carpetas y nada más: `cache/` y `export/`. El resto puede quedarse en solo lectura.

Después conviene lanzar dos guiones por línea de comandos, aunque el sitio funciona sin ellos:

```
php tools/build-search-index.php    # genera data/search-index.json (el buscador)
php tools/build-md-export.php       # pre-genera los .md de export/
```

Qué crea y qué no. No crea tablas, ni usuarios, ni pestañas, ni tareas programadas. Lo único que escribe en disco son ficheros de caché en `cache/` y los Markdown en `export/`.

Qué pasa al desinstalar. Se borra la carpeta y se acabó. No queda nada en ningún sitio.

Si falta el índice de búsqueda, `index.php` intenta generarlo solo en la primera petición (incluye `tools/build-search-index.php`). Funciona, pero la primera visita tarda: mejor lanzarlo a mano.

4. Configuración

Aquí no hay pantalla de configuración: **todo se ajusta en** `config.php`, y es un fichero de constantes. Por eso esta sección no lleva capturas — el proyecto no tiene back-office que capturar.

Lo que hay que tocar sí o sí

Constante	Qué hace de verdad	Qué poner
<code>SITE_DOMAIN</code>	Es el dominio canónico. De aquí salen la etiqueta <code><link rel="canonical"></code> , las URLs del <code>sitemap.xml</code> , las de Open Graph y las del JSON-LD. Si se deja mal, Google indexa el dominio equivocado.	El dominio con <code>https://</code> y sin barra final.
<code>SITE_SUBDIR</code>	Prefijo que se antepone a todos los enlaces internos, a los CSS, al JS y a las llamadas AJAX.	<code>' '</code> si el sitio está en la raíz del dominio. <code>'/docs'</code> si cuelga de una carpeta. Si esto está mal, la página carga sin estilos.

Lo que conviene revisar

Constante	Qué hace de verdad	Nota
<code>CACHE_ENABLED</code>	Con <code>true</code> enciende la caché y además fuerza <code>display_errors=0</code> . Es decir: es también el interruptor de «modo producción». Para depurar hay que ponerlo en <code>false</code> .	Déjalo en <code>true</code> en producción.
<code>CACHE_TTL</code> / <code>CACHE_TTL_SEARCH</code> / <code>CACHE_TTL_STATIC</code>	Segundos que vive cada tipo de entrada: general (1 h), búsquedas (15 min) y contenido estático (24 h).	Con contenido que casi no cambia, se pueden subir sin miedo.
<code>REDIS_HOST</code> / <code>REDIS_PORT</code> / <code>REDIS_DB</code> / <code>REDIS_PREFIX</code>	Solo se usan si la extensión <code>redis</code> está instalada y el servidor responde. Si no, ni se intenta.	El orden de preferencia es Redis → APCu → APC → ficheros en <code>cache/</code> .
<code>GOOGLE_SITE_VERIFICATION</code> / <code>BING_SITE_VERIFICATION</code>	Metén la etiqueta <code><meta name="...-site-verification"></code> en todas las páginas. Vacías, no se pinta nada.	Rellenar solo al dar de alta el sitio en Search Console / Bing.
<code>SEO_DEFAULT_IMAGE</code>	Imagen que sale al compartir un enlace en redes.	Apunta a <code>public/img/og-default.png</code> .
<code>APP_VERSION</code> , <code>APP_NAME</code> , <code>APP_TAGLINE</code> , <code>DEV_NAME</code> , <code>DEV_URL</code>	Textos de marca: cabecera, pie, <code><title></code> , descripción y datos estructurados.	—
<code>SECTIONS</code>	Es la tabla maestra del sitio: cada entrada define el nombre, el icono, el color y la descripción de una sección del menú. Quitar una entrada la borra del menú.	23 secciones. Ver la sección 6.
<code>PS_VERSIONS</code> , <code>PS_VERSION_DEFAULT</code> , <code>PS_VERSION_LATEST</code>	Alimentan los selectores y las etiquetas «Desde 8.1» de las fichas de hook.	—

La clave de la herramienta de caché

`tools/clear-cache.php` vacía OPcache y los ficheros de `cache/`. **Ya no lleva la clave escrita en el código** (ver sección 8): ahora la lee de la variable de entorno `PSDOCS_CACHE_KEY`. Si esa variable no está definida, el acceso por HTTP queda cerrado y el guion solo se puede lanzar por línea de comandos, que es lo normal. Para habilitarlo por web:

```
# En el VirtualHost de Apache
SetEnv PSDOCS_CACHE_KEY "una-cadena-larga-y-aleatoria"
```

y después `https://midominio.com/tools/clear-cache.php?key=una-cadena-larga-y-aleatoria` — teniendo en cuenta que la regla del `.htaccess` bloquea `tools/*.php`, así que hay que quitar esa línea a conciencia si de verdad se quiere abrir. **Lo recomendable es no abrirlo** y usar `php tools/clear-cache.php` por SSH.

El fichero `.user.ini`

Va aparte de `config.php` y ajusta PHP para este sitio: OPcache encendido con 128 MB, caché de rutas reales, `expose_php=0`, `max_execution_time=30`, `memory_limit=64M` y las cookies de sesión marcadas como `httponly`, `secure` y `strict mode`. Solo lo lee PHP-FPM/CGI; con `mod_php` hay que llevar esos valores al `php.ini`.

5. Cómo se usa, día a día

Publicar una guía nueva

Una guía es **un fichero JSON** en `data/guides/{sección}/{slug}.json`. En cuanto el fichero existe, la URL `/ {sección} / {slug}` funciona: no hay que registrar nada en ningún sitio.

El JSON tiene tres partes: `title`, `toc` (el índice de la derecha) y `body`, que es una lista de bloques. Los tipos de bloque que el renderizador entiende están en `src/Renderer/ContentRenderer.php` y son estos, ni uno más:

`heading` · `paragraph` · `code` · `alert` · `table` · `tab-group` · `checklist` · `hook-ref` · `card-grid` · `separator`

Un bloque con un `type` que no esté en esa lista **se ignora en silencio**: no da error, no avisa, simplemente no sale. Es la primera cosa que mirar si un trozo de una guía no aparece.

Dentro de los textos se puede usar Markdown ligero: `**negrita**`, `*cursiva*`, ``código`` y `[texto](url)`. También se admite HTML directo (`<strong>`, `<code>` ...) y de hecho el contenido actual lo usa.

Después de añadir o cambiar guías:

```
php tools/build-search-index.php    # para que salgan en el buscador
php tools/build-md-export.php       # para regenerar los .md (--force para rehacerlos todos)
php tools/clear-cache.php           # para tirar la caché
```

Publicar una novedad

Hay un generador dedicado, y es interactivo:

```
php tools/add-novedad.php
php tools/add-novedad.php --title "PrestaShop 9.2 disponible" --category release --date 2026-06-15
```

Crea el JSON con el esqueleto (`Resumen`, `Detalles`, `Impacto`) y reconstruye el índice de búsqueda solo. Después hay que editar el JSON a mano para rellenar el contenido. Las categorías válidas son `release`,

`security`, `monthly`, `live-update`, `community`, `development` y `announcement`; cada una tiene su icono y su color.

Añadir un hook

Dos ficheros posibles. El listado grande vive en `data/hooks/index.json` (212 hooks) y basta con añadir una entrada ahí para que el hook salga en el buscador y tenga página propia. Si se quiere la ficha completa — parámetros, ejemplo de código, notas, historial de versiones— se crea además `data/hooks/detail/{nombreDelHook}.json`. Ahora mismo 28 de los 212 tienen ficha completa; el resto se pinta con lo que hay en el índice.

Las tres herramientas interactivas

Son páginas que funcionan enteras en el navegador, sin enviar nada al servidor:

- `/tools/scaffold` — Generador de módulos. Se rellena un formulario (nombre, autor, versión, hooks, tabla de base de datos) y devuelve el fichero principal del módulo, el árbol de carpetas y el SQL, listos para copiar o descargar.
- `/tools/hook-explorer` — Explorador visual: se elige una página de PrestaShop y muestra qué hooks se disparan en ella y en qué orden.
- `/tools/compatibility` — Tabla de qué hook existe en qué versión (1.6, 1.7, 8.x, 9.x), para saber de un vistazo si algo se puede usar.

El buscador

Hay dos, y funcionan distinto:

- El **rápido** (⌘K / Ctrl+K) se descarga el índice entero una vez y busca en el navegador: es instantáneo y navega con las flechas. Guarda las últimas búsquedas en el almacenamiento local del navegador.
- La **página** `/search?q=...` llama a `/api/search.php`, que busca en el servidor y guarda el resultado en caché 15 minutos.

Los dos leen `data/search-index.json` (376 entradas ahora mismo). **Si el buscador no devuelve nada, casi siempre es que falta ese fichero:** se regenera con `php tools/build-search-index.php`.

6. Qué ve el visitante

El sitio se organiza en **23 secciones**, todas declaradas en la constante `SECTIONS` de `config.php`. Cada una tiene su página índice con tarjetas y sus páginas de detalle:

Sección	URL	Guías	Qué cubre
Hooks	/hooks	212 hooks	Action, display y filter; ficha por hook con parámetros y ejemplo
Módulos	/modules	13	Estructura, controladores, formularios, routing, traducción, testing
Temas	/themes	8	Smarty, assets, child themes, Hummingbird de PS 9
Overrides	/overrides	6	Clases, controladores, conflictos y alternativas
Base de Datos	/database	7	ObjectModel, DbQuery, esquemas, rendimiento, seguridad
Panel Admin	/admin	7	HelperForm, HelperList, HelperOptions, Grid, KPIs
Front Office	/front	7	FrontController, Context, AJAX, clase Link, Media
Web Services	/webservice	6	Recursos REST, filtrado, errores, ampliación
Seguridad	/security	6	Tokens CSRF, SQL injection, XSS, subidas, permisos, OWASP
Rendimiento	/performance	7	Caché, CCC, imágenes, consultas, perfilado
Multitienda	/multistore	6	Contexto, configuración, consultas, URLs
Traducción	/i18n	6	<code>trans()</code> , ficheros <code>.xlf</code> , dominios, Twig, CLDR
Pagos	/payment	6	Módulo de pago, estados, validación, reembolsos, multidisiva
Transportistas	/carriers	6	Módulo de transporte, cálculo, zonas, seguimiento, puntos de recogida
Emails	/email	6	Plantillas, <code>Mail::Send()</code> , SMTP, hooks de correo
Trucos & Skills	/trucos	18	Patrones avanzados: Grids, Doctrine, pestañas de producto, overrides blindados
Guía Back Office	/bo-guide	8	Manual del administrador: pedidos, productos, clientes, pagos, transporte
Guía Front Office	/fo-guide	6	Catálogo, checkout, cuenta de cliente, búsqueda, CMS, contacto
Catálogo	/catalog	6	Categorías, atributos, características, precios, stock, importar/exportar
Ejemplos reales	/examples	22	Código completo de módulos: AJAX, cron, CSV, GTM, Doctrine, webservice
PrestaShop 9.1	/ps91	8	Cambios de la 9.1: multi-envío, descuentos, Hummingbird, política de contraseñas

Sección	URL	Guías	Qué cubre
Módulos Ejemplo PS	<code>/sample-modules</code>	7	Los módulos oficiales de ejemplo de PrestaShop
Novedades 2026	<code>/novedades</code>	7	Releases, avisos de seguridad, Core Monthly, eventos

Total actual: **179 guías** repartidas en esas secciones, más las 212 fichas de hook.

Además de eso, el visitante encuentra:

- **Índice lateral** en cada página (la tabla de contenidos de la derecha), con enlaces a cada apartado.
- **Botón «Copiar»** en cada bloque de código.
- **Descarga en Markdown**: en las páginas que tienen JSON detrás aparece un botón que enlaza a `/export/{sección}/{slug}.md`. Son 207 ficheros pre-generados; si se pide uno que no existe, PHP lo genera al vuelo y lo guarda para la próxima. Están pensados para pegárselos a un asistente de IA.
- `/sitemap.xml` con todas las URLs y `/robots.txt` que permite `/export/` y prohíbe `/api/`, `/cache/`, `/data/`, `/src/` y `/tools/`.
- **Datos estructurados** JSON-LD (`WebSite`, `BreadcrumbList`, `TechArticle`) y etiquetas Open Graph y Twitter Card en todas las páginas.

7. Preguntas frecuentes y problemas

Todos los enlaces dan 404 menos la portada. Es `mod_rewrite`. O no está activo, o Apache tiene `AllowOverride None` para esa carpeta y no lee el `.htaccess`. En Nginx, la configuración equivalente está comentada al final del propio `.htaccess`.

La página carga sin estilos, todo en blanco y negro. `SITE_SUBDIR` no coincide con dónde está instalado el sitio. Si cuelga de `/docs`, tiene que poner `'/docs'`, y el `.htaccess` tiene que llevar `RewriteBase /docs`.

El buscador no encuentra nada. Falta `data/search-index.json`. Se genera con `php tools/build-search-index.php`. Si el fichero está y aun así no sale nada, mirar en la consola del navegador si `/api/search-index.php` devuelve 200: es el endpoint del que el buscador rápido se descarga el índice.

He cambiado un JSON y la web sigue enseñando lo de antes. Es la caché, y hay tres capas. La de fichero/Redis (`cache/`), OPcache, y la del navegador. Se limpian todas con `php tools/clear-cache.php`. Ojo: los `.md` de `export/` son ficheros estáticos y no se regeneran solos si ya existen — hay que pasar `php tools/build-md-export.php --force`.

Un trozo de una guía no aparece. El `type` del bloque no es ninguno de los diez que el renderizador conoce (ver sección 5). Los tipos desconocidos se ignoran sin dar error.

Una entrada del índice lateral sale en blanco. Ya no. Era un fallo real: los JSON usan unas veces `text` y otras `title` para el texto de cada entrada, y la plantilla solo leía `text`, así que **153 entradas del índice salían vacías** y además llenaban el log de avisos. Corregido (ver sección 8).

¿Se puede alojar en un hosting compartido barato? Sí. No hay base de datos, ni Composer, ni procesos de fondo. Con PHP 8 y `mod_rewrite` sobra. Redis y APCu son opcionales.

¿Cómo se traduce a otro idioma? No se puede sin trabajo de desarrollo. El sitio es monolingüe por diseño: los textos están en los JSON y en las plantillas, sin sistema de traducción.

¿Se puede publicar contenido desde el navegador? No, y es deliberado. No hay panel de administración, ni formularios de escritura, ni login. Todo el contenido entra por fichero y por línea de comandos. Esa es justamente la razón de que la superficie de ataque sea tan pequeña.

8. Seguridad

Revisión completa del código: `index.php`, `config.php`, los cinco ficheros de `src/`, las plantillas, los tres endpoints de `api/`, las 23 páginas de `public/pages/`, los cuatro guiones de `tools/`, el JavaScript de `public/js/` y el `.htaccess`.

Lo que se ha arreglado en esta revisión

1. Travesía de directorios al servir ficheros estáticos — `index.php` `index.php` servía a mano los `.css`, `.js`, imágenes y tipografías concatenando la ruta pedida con `__DIR__`, sin comprobar nada más que la extensión. Como la ruta se puede meter directamente por `?route=`, valía `?route=../..loquesea.js` para leer ficheros de fuera del directorio del sitio. Y el filtro de carpetas prohibidas (`data`, `src`, `cache`) solo miraba el principio de la cadena, así que `public/../src/Bootstrap.php.js` se lo saltaba. Ahora la ruta se normaliza (fuera el byte nulo, las barras invertidas a barras normales) y se rechaza cualquier cosa con `..`, y además el fichero servido tiene que colgar de verdad del directorio del sitio, comprobado con `realpath()`. Verificado: los cuatro intentos de travesía que se probaron devuelven 403.

2. Clave de administración escrita en el código — `tools/clear-cache.php` La herramienta que vacía la caché llevaba la contraseña dentro del fichero (`define('CACHE_CLEAR_KEY', 'psdocs2024clear')`), a la vista de cualquiera con acceso al repositorio, y comparaba con `!==`, que filtra la clave carácter a carácter midiendo tiempos. Ahora sale de la variable de entorno `PSDOCS_CACHE_KEY`, se compara con `hash_equals()` (que tarda siempre lo mismo) y, si la variable no está definida, el acceso por HTTP queda cerrado del todo. Por línea de comandos sigue funcionando sin clave.

3. Guiones de mantenimiento accesibles por HTTP — `tools/`, `.htaccess` De los cuatro guiones de `tools/`, solo uno estaba bloqueado y solo uno comprobaba que se le llamaba desde la línea de comandos. Ahora `tools/add-novedad.php` y `tools/build-search-index.php` cortan si la petición no es de consola —el segundo con una excepción, porque `index.php` lo incluye de forma legítima cuando falta el índice de búsqueda— y el `.htaccess` bloquea todo `tools/*.php`. **Cuidado con esta regla:** se bloquea *por extensión* `.php`, no la ruta entera, porque `/tools/scaffold`, `/tools/hook-explorer` y `/tools/compatibility` son **páginas públicas del sitio**. Bloquear `^tools(/.*)?$` las tumba las tres. Se detectó precisamente probando las páginas una a una.

4. Documentación interna servida y indexable — `.htaccess` `INSTALL.md` y `PLAN_COMPLETO.md` (107 KB de notas internas del proyecto) estaban en la raíz y se servían tal cual; peor aún, la regla de cabeceras de los

`.md` les ponía `X-Robots-Tag: index, follow`, o sea que se invitaba a Google a indexarlas. Bloqueados los `.md` de la raíz. Los de `export/`, que son los públicos, siguen accesibles.

5. Ficheros de arranque servibles — `.htaccess` `config.php`, `opcache-preload.php` y `.user.ini` no estaban bloqueados. No filtran nada por sí solos (no imprimen), pero no hay ninguna razón para servirlos. Bloqueados.

6. HTML sin escapar en el renderizador — `src/Renderer/ContentRenderer.php` Cuatro puntos donde un dato del JSON se pintaba en crudo dentro de HTML: - El **nivel del encabezado** iba directo al nombre de la etiqueta (`<h{$level}>`). Ahora se convierte a entero y se acota a 1-6. - El **id del encabezado** se metía sin escapar en dos atributos. Escapado. - El **nombre de fichero** y el **título** de los bloques de código, sin escapar. Escapados. - Los **enlaces** `[texto](url)` aceptaban cualquier esquema, incluido `javascript:`. Ahora solo pasan `http://`, `https://`, `mailto:`, las rutas relativas y los anclajes; lo demás se sustituye por `#`. Comprobado que los enlaces reales siguen funcionando y que `[x](javascript:alert(1))` sale neutralizado.

El texto de los párrafos **sigue admitiendo HTML a propósito**: el contenido actual lo usa (hay `<strong>` dentro de los textos de las guías). Es contenido escrito por quien mantiene el sitio, no por visitantes, así que se trata como de confianza. Queda dicho en «Lo que queda pendiente».

7. XSS reflejado en la página de búsqueda — `public/pages/search.php` Los resultados se montaban con `innerHTML` concatenando `r.title`, `r.desc`, `r.url` y `r.section` sin escapar, y el término buscado se inyectaba en el mensaje de «sin resultados». Se ha añadido una función `esc()` y pasa por ella todo lo que entra en `innerHTML`. Además, el `json_encode()` que pasa el término buscado al JavaScript ahora lleva `JSON_HEX_TAG|JSON_HEX_AMP|JSON_HEX_APOS|JSON_HEX_QUOT`. Verificado con una carga `<img src=x onerror=alert(1)>`: sale escapada.

8. `unserialize()` sin restringir clases — `src/Cache/Cache.php` Los dos puntos donde se lee la caché (fichero y Redis) hacían `unserialize()` sin límites. La caché solo guarda arrays y escalares, así que se ha añadido `['allowed_classes' => false]`: si alguien lograra escribir en `cache/` o en Redis, ya no puede instanciar objetos.

9. Robustez PHP 8 — `api/hook.php`, `tools/` `api/hook.php` recorría el índice leyendo `$h['id']` sin comprobar que la clave existiera, y sin comprobar que lo devuelto fuera un array. Los `glob()` de los guiones de `tools/` se recorrían directamente con `foreach`, y `glob()` devuelve `false` cuando falla — en PHP 8 eso es un `TypeError`. Añadidas las guardas en los dos sitios.

Fallos de funcionamiento encontrados y arreglados

No eran fallos de seguridad, pero rompían el sitio y salieron de la misma revisión:

10. El buscador estaba muerto entero. Dos fallos independientes, cada uno de los cuales bastaba para dejarlo sin funcionar:

- `api/search.php` cargaba `config.php` y las clases a mano, pero **no cargaba Bootstrap**, que es de quien depende `SearchEngine` para leer el índice. Resultado: `Uncaught Error: Class "Bootstrap" not found` y un **500 en cada consulta**. Reproducido y arreglado; el endpoint devuelve ahora los resultados correctos.
- El buscador rápido (⌘K) se descargaba el índice de `/data/search-index.json`, una ruta que **el propio .htaccess bloquea con un 403**. El `fetch` fallaba, el `catch` dejaba el índice vacío y el buscador no

devolvía nunca nada, sin un solo error visible. Se ha añadido el endpoint `api/search-index.php`, que sirve el índice con su `Etag` y su caché, y `public/js/search.js` apunta ahí.

11. 153 entradas del índice lateral salían en blanco. Los JSON de `data/` usan `text` en unos sitios y `title` en otros para el texto de cada entrada del índice; la plantilla `src/templates/partials/toc.php` solo leía `text`. Contadas: 897 entradas con `text` y **153 con `title`**, que salían vacías y además soltaban un `Undefined array key` y un `Deprecated: htmlspecialchars(): Passing null` por cada una. La plantilla acepta ahora las dos claves.

12. Aviso de PHP en todas las peticiones. `index.php` definía `DOCR00T` y `config.php` volvía a definirla: `Warning: Constant DOCR00T already defined` en cada carga. En producción solo ensuciaba el log, pero es un defecto. Corregido con `if (!defined(...))`.

13. Rutas absolutas en el JavaScript. `public/js/app.js` llamaba a `/api/hook.php` con barra inicial, ignorando `BASE_PATH`, y `public/pages/search.php` hacía lo mismo con `/api/search.php`. En una instalación en subdirectorio las dos llamadas fallaban. Ahora las dos usan el prefijo configurado.

14. Requisito de PHP mal documentado. El `INSTALL.md` decía «PHP 7.4+» y el código no arranca en 7.4 (usa `match()` y `str_contains()`). Corregido en el propio `INSTALL.md`.

Lo que se ha revisado y estaba bien

- **Inyección SQL:** no aplica, no hay base de datos. Ni una consulta en todo el proyecto.
- **Ejecución de código:** no hay `eval`, `exec`, `shell_exec`, `system` ni `passthru` en ninguna parte.
- **Peticiones a terceros:** el sitio no llama a ningún servicio externo. No hay `curl`, ni `file_get_contents()` remoto, ni webhooks. El único `curl_*` que aparece es texto de ejemplo dentro de una guía.
- **Subida de ficheros:** no hay ninguna. Ni un `$_FILES` en el proyecto.
- **Sesiones y autenticación:** no hay. No hay login, ni cookies propias, ni datos de usuario, ni panel de administración. Por eso tampoco hace falta CSRF: no existe ninguna acción autenticada que proteger.
- **Datos personales:** el sitio no recoge ninguno. No hay formularios de envío.
- **Barrido masivo sobre histórico:** no aplica. No hay bucles que manden correos, mensajes ni notificaciones a nadie; no hay histórico que recorrer.
- **Validación de los parámetros de URL:** correcta ya de origen. Todos los *slugs* (`hooks/detail.php`, `modules/detail.php`, `trucos/detail.php`, `novedades-detail.php`, `generic-section-router.php`, `api/hook.php`) pasan por un `preg_replace` que deja solo `[a-zA-Z0-9_-]` antes de tocar el sistema de ficheros. El endpoint de exportación a Markdown valida con una expresión regular igual de estricta.
- **Cabeceras de seguridad:** el `.htaccess` ya traía `X-Content-Type-Options`, `X-Frame-Options`, `Referrer-Policy`, `Permissions-Policy` y una `Content-Security-Policy` razonable.
- **Escapado en el buscador rápido:** `public/js/search.js` ya tenía su función `esc()` y la usaba en todos los puntos, y el resaltado de términos escapa la expresión regular antes de usarla. Bien hecho de origen.
- **Generador de módulos:** `public/js/tools/scaffold.js` funciona entero en el navegador, no envía nada al servidor, y filtra el nombre del módulo a `[a-z0-9_]`.

Lo que queda pendiente

- **El contenido de `data/` se trata como de confianza.** Los textos de los párrafos, las celdas de las tablas y los elementos de las listas admiten HTML a propósito, porque el contenido actual lo usa. Mientras solo escriba en `data/` quien administra el servidor, no hay riesgo. **Si alguna vez se abre la publicación a terceros, esto hay que cambiarlo antes**, y no es un cambio pequeño: habría que definir una lista blanca de etiquetas y repasar las 179 guías.
- **Unas cuarenta salidas sin escapar en las plantillas** (`public/pages/*.php`), del tipo `<?= $sec['icon'] ?>` o `<?= $cat['color'] ?>`. Todas leen datos locales, ninguna llega desde la URL: se han comprobado una a una. Se han dejado como están porque escaparlas rompería los iconos y los colores, y el riesgo real es cero mientras valga el punto anterior. Escaparlas correctamente es trabajo de una tarde y merece la pena hacerlo antes de abrir el contenido a terceros.
- **Access-Control-Allow-Origin: *** en `api/search.php`. Cualquier sitio puede consultar la API de búsqueda desde el navegador. Como el contenido es público y no hay ni sesión ni datos personales, no filtra nada; se ha dejado a propósito porque es un endpoint pensado para consumirse desde fuera. Si algún día se quiere cerrar, basta con quitar esa cabecera.
- **Ortografía castellana: 8.151 palabras sin tilde o sin ñe** en los contenidos («documentacion», «modulos», «busqueda», «configuracion», «anadir...»), repartidas por los JSON de `data/` y por las plantillas. Es contenido de cara al público y conviene arreglarlo, pero es un cambio masivo sobre 485 ficheros de texto y con las tildes diacríticas (`solo` / `sólo` , `esta` / `está`) hay que repasar a mano. **No se ha tocado: hace falta el visto bueno antes de lanzarlo.**
- **Sin capturas de pantalla.** El encargo excluía el paso de capturas por no ser un módulo de PrestaShop. Este manual no lleva imágenes por eso.

Cómo se ha comprobado

El sitio se levantó en local con el servidor de PHP 8.2 y `error_reporting=E_ALL` y se recorrieron, comprobando el código de respuesta, el tamaño de la página y el registro de errores:

- las 23 páginas índice de sección,
- las 179 páginas de guía, una por una,
- las 28 fichas de hook con detalle, más 40 fichas sacadas solo del índice,
- las 28 exportaciones a Markdown,
- las tres herramientas interactivas,
- los tres endpoints de `api/`, incluidos los casos de error (falta el `id`, `id` que no existe),
- `/sitemap`, `/robots`, la página 404 y la de búsqueda,
- y los intentos de travesía de directorios y de acceso a `data/`, `src/`, `cache/` y a los guiones de `tools/`.

Resultado: unas 300 URLs, 0 fallos, 0 avisos de PHP y 0 obsolescencias. Todos los ficheros PHP pasan `php -l` en 8.2 y en 8.4; los dos `.js` tocados pasan `node --check`.

9. Ficha técnica

Estructura

ayudaps/	
index.php	Punto de entrada. Enrutado, ficheros estáticos, exportación a Markdown, bloqueo de carpetas
config.php	Todas las constantes de configuración
opcache-preload.php	Precarga de OPcache (opcional, se pone en php.ini)
.htaccess	Reescrituras, cabeceras de seguridad, compresión, bloqueos
.user.ini	Ajustes de PHP para este sitio
INSTALL.md	Instalación (uso interno, bloqueado por HTTP)
PLAN_COMPLETO.md	Notas del proyecto (uso interno, bloqueado por HTTP)
src/	
Bootstrap.php	Arranque, autocarga por mapa de clases, caché de JSON
Router/Router.php	Tabla de rutas y resolución de patrones {slug}
Cache/Cache.php	Caché: Redis → APCu → APC → ficheros
Renderer/ContentRenderer.php	Convierte los bloques del JSON en HTML
Search/SearchEngine.php	Búsqueda y puntuación en el servidor
Export/MarkdownExporter.php	Convierte guías y hooks a Markdown
templates/layout.php	Plantilla principal: SEO, JSON-LD, cabecera, pie
templates/partials/	sidebar-nav.php (menú), toc.php (índice lateral)
public/	
pages/	23 páginas PHP (una por tipo de vista)
css/	13 hojas de estilo
js/	app.js, search.js, hooks.js, scaffold.js, request-builder.js, background-stream.js
api/	
search.php	Búsqueda en el servidor (JSON)
search-index.php	Sirve el índice para el buscador rápido (nuevo)
hook.php	Ficha de un hook en JSON
data/	TODO el contenido. Bloqueada por HTTP
hooks/index.json	212 hooks
hooks/detail/*.json	28 fichas completas
guides/{sección}/*.json	179 guías en 23 secciones
search-index.json	376 entradas (generado)
classes/ compatibility/ schemas/ webservice/ database/tables/	vacías, reservadas
export/	207 .md pre-generados. Públicos a propósito
cache/	Caché en fichero. Bloqueada por HTTP
tools/	4 guiones. Solo línea de comandos

Rutas públicas

Patrón	Página
/	Portada
/ {sección}	Índice de sección (23 secciones)
/ {sección} / {slug}	Guía
/ hooks / {nombreDelHook}	Ficha de hook
/ tools / scaffold , / tools / hook-explorer , / tools / compatibility	Herramientas
/ search ? q = ...	Búsqueda
/ export / {sección} / {slug} . md	Versión Markdown
/ sitemap . xml , / robots . txt	SEO
/ api / search . php , / api / search-index . php , / api / hook . php	Endpoints JSON

Rutas bloqueadas

`data/` , `src/` , `cache/` (por `.htaccess` y por `index.php`) , `tools/*.php` , `config.php` , `opcache-preload.php` , `.user.ini` , los `.md` de la raíz , `.git` , `.env` y `composer.json` / `composer.lock` .

Base de datos, cron y permisos

- **Tablas:** ninguna. No hay base de datos.
- **Tareas programadas:** ninguna obligatoria. Si el contenido se actualiza a menudo, conviene un cron con `php tools/build-search-index.php && php tools/build-md-export.php` .
- **Permisos de escritura:** solo `cache/` y `export/` .
- **Registros propios:** no hay carpeta `logs/` . Los errores de PHP van al log del servidor; `config.php` deja `display_errors=0` y `log_errors=1` .

Guiones de línea de comandos

Guion	Qué hace
<code>php tools/build-search-index.php</code>	Regenera <code>data/search-index.json</code> a partir de hooks y guías
<code>php tools/build-md-export.php</code>	Genera los <code>.md</code> de <code>export/</code> . <code>--force</code> rehace todo, <code>--clean</code> vacía primero
<code>php tools/add-novedad.php</code>	Crea el JSON de una novedad. Interactivo o con <code>--title</code> , <code>--slug</code> , <code>--category</code> , <code>--date</code> , <code>--desc</code>
<code>php tools/clear-cache.php</code>	Vacía OPcache, la caché de rutas y los ficheros de <code>cache/</code>

Ficheros tocados en esta revisión

Fichero	Motivo
<code>index.php</code>	Travesía de directorios, normalización de la ruta pedida
<code>config.php</code>	<code>DOCROOT</code> redefinida
<code>.htaccess</code>	Bloqueo de <code>tools/*.php</code> , ficheros de arranque y <code>.md</code> de la raíz
<code>api/search.php</code>	Cargaba mal las dependencias: 500 en cada consulta
<code>api/search-index.php</code>	Nuevo. Sirve el índice al buscador rápido
<code>api/hook.php</code>	Guardas de PHP 8
<code>src/Renderer/ContentRenderer.php</code>	Escapado de HTML y filtro de esquemas en los enlaces
<code>src/Cache/Cache.php</code>	<code>unserialize()</code> sin clases
<code>src/Search/SearchEngine.php</code>	Puntuación de títulos que no coincidía nunca
<code>src/templates/partials/toc.php</code>	153 entradas del índice en blanco
<code>public/pages/search.php</code>	XSS reflejado, <code>BASE_PATH</code> en la llamada AJAX
<code>public/js/search.js</code>	Índice de búsqueda desde el endpoint nuevo
<code>public/js/app.js</code>	<code>BASE_PATH</code> en la llamada AJAX
<code>tools/clear-cache.php</code>	Clave fuera del código, comparación en tiempo constante
<code>tools/add-novedad.php</code>	Solo por línea de comandos
<code>tools/build-search-index.php</code>	Solo por línea de comandos, constantes sin redefinir
<code>INSTALL.md</code>	Requisito de PHP corregido a 8.0

AyudaPS — documentación NO oficial de PrestaShop. Desarrollado por gmartos.es.